

EJERCICIO PUNTUAL 2026-I

DANIEL ALEJANDRO CHAVEZ BUSTOS - 20212020109

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS
FACULTAD DE INGENIERÍA
DISEÑO ARQUITECTURAL DE SOFTWARE Y PATRONES
BOGOTÁ D.C
2026
EJERCICIO PUNTUAL

CONTRAPORTADA

Fecha	Revisor	Revisión	Observación	Aprobó
26/04/2026	Daniel Alejandro Chavez Bustos	Primer punto del ejercicio puntual (transformación de código a Python)		Sí
26/04/2026	Daniel Alejandro Chavez Bustos	Segundo Punto del ejercicio puntual (Recuperación de diseño)		Sí
30/04/2026	Daniel Alejandro Chavez Bustos	Revisión de formato, redacción		Sí

Versión: 1.0

TABLA DE CONTENIDO

	Pág.
1 INTRODUCCIÓN	5
2 TRANSFORMACIÓN DE CÓDIGO ORIGINAL	5
2.1 DESCRIPCIÓN DE PLATAFORMAS DE IA GENERATIVA, PROGRAMACIÓN Y EJECUCIÓN DE PROGRAMAS	5
2.2 “PROMPT” UTILIZADO PARA LA TRANSFORMACIÓN DE CÓDIGO	6
2.3 CÓDIGO TRANSFORMADO (PYTHON)	18
2.4 ANÁLISIS DE RESULTADO OBTENIDO	25
2.5 REFLEXIÓN FINAL	26
3 RECUPERACIÓN DE DISEÑO	27
3.1 DESCRIPCIÓN DE PLATAFORMAS DE IA GENERATIVA, PROGRAMACIÓN Y EJECUCIÓN DE PROGRAMAS	27
3.2 “PROMPT” UTILIZADO PARA LA TRANSFORMACIÓN DE CÓDIGO	27
3.4 ANÁLISIS COMPARATIVO DE LOS MODELOS ENTREGADOS DE LA IA	41
3.4.1 DIAGRAMA FUNCIONAL	41
3.4.2 DIAGRAMA ESTRUCTURAL	42
3.4.3 DIAGRAMA DE SECUENCIA	44
3.5 REFLEXIÓN FINAL	45

LISTA DE FIGURAS

	Pág
Figura 1: GUI en Python	25
Figura 2: Diagramas de Casos de Uso Propuesto por la	41
Figura 3: Diagramas de Casos de Uso Creado en la Asignatura	41
Figura 4: Diagramas de Clases Generado por la IA	42
Figura 5: Diagramas de Clases Creado en la Asignatura	43
Figura 6: Diagrama de Secuencia generado por la IA/ CU “search vehicle”	44
Figura 7: Diagrama de Secuencia generado por la IA/ CU “Exit Application”	44

1 INTRODUCCIÓN

El presente documento tiene como propósito desarrollar un ejercicio práctico en el marco del diseño arquitectónico de software y el uso de patrones de diseño, específicamente a través de la implementación y análisis del patrón Fábrica Abstracta. Para ello, se parte de un sistema desarrollado en lenguaje Java, el cual es sometido a un proceso de transformación a lenguaje Python utilizando herramientas de inteligencia artificial generativa.

Este trabajo se enfoca no solo en la conversión del código entre lenguajes, sino también en la evaluación de la capacidad de las herramientas de IA para comprender, adaptar y mantener estructuras de diseño orientadas a objetos, preservando la lógica y funcionalidad original del sistema. En este sentido, se documenta las herramientas usadas, el prompt utilizado, el código generado y un análisis crítico de los resultados obtenidos.

Adicionalmente, se reflexiona sobre el impacto del uso de inteligencia artificial en tareas de desarrollo de software, particularmente en escenarios como la migración de tecnologías y la recuperación de diseño a partir de un código existente, resaltando tanto sus ventajas en términos de eficiencia como sus limitaciones y la necesidad de supervisión técnica.

2 TRANSFORMACIÓN DE CÓDIGO ORIGINAL

2.1 DESCRIPCIÓN DE PLATAFORMAS DE IA GENERATIVA, PROGRAMACIÓN Y EJECUCIÓN DE PROGRAMAS

El modelo de inteligencia artificial utilizado durante el desarrollo del ejercicio fue DeepSeek-V3.2, una versión puesta en producción en diciembre de 2025 que representa una evolución dentro de la línea de modelos desarrollados por la misma empresa. Esta versión se basa en una arquitectura MoE (Mixture of

Experts, o Mezcla de Expertos), característica de DeepSeek, diseñada para optimizar el desempeño en tareas especializadas.

El modelo ha sido concebido principalmente para actividades de razonamiento, generación de código y análisis de textos extensos. Una de sus capacidades más destacadas es la gestión de una ventana de contexto de hasta 128 mil tokens en su capa gratuita, lo que le permite procesar grandes volúmenes de información y mantener coherencia lógica y narrativa incluso en escenarios complejos, como el análisis de código a gran escala en una única instancia de memoria.

En la práctica, se empleó el modo experto de la plataforma, el cual, según sus especificaciones, está orientado a la resolución de tareas de mayor complejidad, lo que resultó adecuado para los objetivos planteados en el ejercicio.

La plataforma de programación utilizada es Visual Studio Code 1.115.0.

La version de los programas de ejecución de código son Java “17.0.9” y Python “3.10.4” respectivamente.

2.2 “PROMPT” UTILIZADO PARA LA TRANSFORMACIÓN DE CÓDIGO

You are an AI assistant tasked with translating code from Java to Python. Your goal is to produce clean, efficient, and well-structured Python code that preserves the original logic while following Pythonic best practices.

Here is the Java source code to be translated:

```
<java_code>
  <project_structure>
    AutoSearchUI.java
    CARVehicleFactory.java
    HATCHBACKVehicleFactory.java
    Luxury.java
    LuxuryCAR.java
    LuxuryHATCHBACK.java
    LuxurySUV.java
    NonLuxury.java
    NonLuxuryCAR.java
    NonLuxuryHATCHBACK.java
```

```

        NonLuxurySUV.java
        SUVVehicleFactory.java
        VehicleFactory.java
    </project_structure>
    <files>
        <file path="AutoSearchUI.java">
            <![CDATA[
import java.io.*;
import java.io.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import com.sun.java.swing.plaf.windows.*;

public class AutoSearchUI extends JFrame {
    public static final String newline = "\n";
    public static final String SEARCH = "Search";
    public static final String EXIT = "Exit";
    public static final String LUXURY = "Luxury";
    public static final String NON_LUXURY = "Non-Luxury";

    private JComboBox cmbVehicleCategory, cmbVehicleType;

    private JLabel lblVehicleCategory, lblVehicleType,
        lblCarName, lblCarNameValue;

    public AutoSearchUI() {
        super("Abstract Factory - Example");

        cmbVehicleCategory = new JComboBox();
        cmbVehicleCategory.addItem(AutoSearchUI.LUXURY);
        cmbVehicleCategory.addItem(AutoSearchUI.NON_LUXURY);

        cmbVehicleType = new JComboBox();
        cmbVehicleType.addItem(VehicleFactory.CAR);
        cmbVehicleType.addItem(VehicleFactory.SUV);
        cmbVehicleType.addItem(VehicleFactory.HATCHBACK);

        lblVehicleCategory = new JLabel("Vehicle Category:");

```

```

lblVehicleType = new JLabel("VehicleType:");
lblCarName = new JLabel("Search Result:");
lblCarNameValue = new JLabel(" Please click on Search button");

// Create the open button
JButton openButton = new JButton(AutoSearchUI.SEARCH);
openButton.setMnemonic(KeyEvent.VK_S);
JButton exitButton = new JButton(AutoSearchUI.EXIT);
exitButton.setMnemonic(KeyEvent.VK_X);

ButtonHandler objButtonHandler = new ButtonHandler(this);

openButton.addActionListener(objButtonHandler);
exitButton.addActionListener(new ButtonHandler());

JPanel buttonPanel = new JPanel();

// *****
GridBagLayout gridbag = new GridBagLayout();
buttonPanel.setLayout(gridbag);
GridBagConstraints gbc = new GridBagConstraints();

buttonPanel.add(lblVehicleCategory);
buttonPanel.add(cmbVehicleCategory);
buttonPanel.add(lblVehicleType);
buttonPanel.add(cmbVehicleType);
buttonPanel.add(lblCarName);
buttonPanel.add(lblCarNameValue);

buttonPanel.add(openButton);
buttonPanel.add(exitButton);

gbc.insets.top = 5;
gbc.insets.bottom = 5;
gbc.insets.left = 5;
gbc.insets.right = 5;

gbc.anchor = GridBagConstraints.EAST;
gbc.gridx = 0;

```



```

gbc.gridy = 0;
gridbag.setConstraints(lblVehicleCategory, gbc);
gbc.anchor = GridBagConstraints.WEST;

gbc.gridx = 1;
gbc.gridy = 0;
gridbag.setConstraints(cmbVehicleCategory, gbc);
gbc.anchor = GridBagConstraints.EAST;
gbc.gridx = 0;
gbc.gridy = 1;
gridbag.setConstraints(lblVehicleType, gbc);
gbc.anchor = GridBagConstraints.WEST;
gbc.gridx = 1;
gbc.gridy = 1;
gridbag.setConstraints(cmbVehicleType, gbc);
gbc.anchor = GridBagConstraints.EAST;
gbc.gridx = 0;
gbc.gridy = 2;
gridbag.setConstraints(lblCarName, gbc);
gbc.anchor = GridBagConstraints.WEST;
gbc.gridx = 1;
gbc.gridy = 2;
gridbag.setConstraints(lblCarNameValue, gbc);

gbc.insets.left = 2;
gbc.insets.right = 2;
gbc.insets.top = 40;
gbc.anchor = GridBagConstraints.EAST;

gbc.gridx = 0;
gbc.gridy = 5;
gridbag.setConstraints(openButton, gbc);
gbc.anchor = GridBagConstraints.WEST;
gbc.gridx = 1;
gbc.gridy = 5;
gridbag.setConstraints(exitButton, gbc);

// *****

```

```

// Add the buttons and the log to the frame
Container contentPane = getContentPane();

contentPane.add(buttonPanel, BorderLayout.CENTER);
try {
    // UIManager.setLookAndFeel(new WindowsLookAndFeel());
    SwingUtilities.updateComponentTreeUI(
        AutoSearchUI.this);
} catch (Exception ex) {
    System.out.println(ex);
}

}

public static void main(String[] args) {
    JFrame frame = new AutoSearchUI();

    frame.addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });

    // frame.pack();
    frame.setSize(1050, 600);
    frame.setVisible(true);
}

public String getSelectedCategory() {
    return (String) cmbVehicleCategory.getSelectedItem();
}

public String getSelectedType() {
    return (String) cmbVehicleType.getSelectedItem();
}

public void setResult(String searchResult) {
    lblCarNameValue.setText(searchResult);
}

```

```

} // End of class AutoSearchUI

class ButtonHandler implements ActionListener {
    AutoSearchUI objAutoSearchUI;

    public ButtonHandler() {
    }

    public ButtonHandler(AutoSearchUI inObjAutoSearchUI) {
        objAutoSearchUI = inObjAutoSearchUI;
    }

    public void actionPerformed(ActionEvent e) {

        String searchResult = null;

        if (e.getActionCommand().equals(AutoSearchUI.EXIT)) {
            System.exit(1);
        }
        if (e.getActionCommand().equals(AutoSearchUI.SEARCH)) {
            // get input values
            String vhCategory = objAutoSearchUI.getSelectedCategory();
            String vhType = objAutoSearchUI.getSelectedType();

            // get one of Luxury or NonLuxury vehicle factories
            VehicleFactory vf = VehicleFactory.getVehicleFactory(vhType);

            if (vhCategory.equals(AutoSearchUI.LUXURY)) {
                Luxury c = vf.getLuxury();
                searchResult = "Name: " + c.getLuxuryName() + " Features: " +
                    c.getLuxuryFeatures();
            }
            if (vhCategory.equals(AutoSearchUI.NON_LUXURY)) {
                NonLuxury s = vf.getNonLuxury();
                searchResult = "Name: " + s.getNLName() + " Features: " +
                    s.getNLFeatures();
            }
            objAutoSearchUI.setResult(searchResult);
        }
    }
}

```

```

    }

} // End of class ButtonHandler
]]>
</file>

<file path="CARVehicleFactory.java">
<![CDATA[
public class CARVehicleFactory extends VehicleFactory {

    public LuxuryCAR getLuxury() {
        return new LuxuryCAR("Luxury-Car");
    }
    public NonLuxuryCAR getNonLuxury() {
        return new NonLuxuryCAR("NonLuxuryL-Car");
    }
} // End of class
]]>
</file>

<file path="HATCHBACKVehicleFactory.java">
<![CDATA[
public class HATCHBACKVehicleFactory extends VehicleFactory {

    public LuxuryHATCHBACK getLuxury() {
        return new LuxuryHATCHBACK("Hatchback-Luxury");
    }
    public NonLuxuryHATCHBACK getNonLuxury() {
        return new NonLuxuryHATCHBACK("Hatchback-NonLuxury");
    }
} // End of class
]]>
</file>

<file path="Luxury.java">
<![CDATA[
public interface Luxury {
    public String getLuxuryName();

```

```

        public String getLuxuryFeatures();

    } // End of class
    ]]>
</file>

```

```

        <file path="LuxuryCAR.java">
        <![CDATA[
public class LuxuryCAR implements Luxury {
    private String name;
    public LuxuryCAR(String cName) {
        name = cName;
    }
    public String getLuxuryName() {
        return name;
    }
    public String getLuxuryFeatures() {
        return "Luxury Car Features ";
    };

} // End of class
    ]]>
</file>

```

```

        <file path="LuxuryHATCHBACK.java">
        <![CDATA[
public class LuxuryHATCHBACK implements Luxury {

    private String name;
    public LuxuryHATCHBACK(String cName) {
        name = cName;
    }
    public String getLuxuryName() {
        return name;
    }
    public String getLuxuryFeatures() {
        return "Luxury HATCHBACK Features ";
    };

```

```
}  
    >  
</file>
```

```
    <file path="LuxurySUV.java">  
    <![CDATA[  
public class LuxurySUV implements Luxury {  
    private String name;  
    public LuxurySUV(String sName) {  
        name = sName;  
    }  
    public String getLuxuryName() {  
        return name;  
    }  
    public String getLuxuryFeatures() {  
        return "Luxury SUV Features ";  
    }  
};  
  
} // End of class  
    >  
</file>
```

```
    <file path="NonLuxury.java">  
    <![CDATA[  
public interface NonLuxury {  
    public String getNLName();  
    public String getNLFeatures();  
  
} // End of class  
    >  
</file>
```

```
    <file path="NonLuxuryCAR.java">  
    <![CDATA[  
public class NonLuxuryCAR implements NonLuxury {  
    private String name;  
    public NonLuxuryCAR(String sName) {  
        name = sName;  
    }  
}
```

```

    public String getNLName() {
        return name;
    }
    public String getNLFeatures() {
        return "Non-Luxury CAR Features ";
    };

} // End of class
]]>
</file>

<file path="NonLuxuryHATCHBACK.java">
<![CDATA[
public class NonLuxuryHATCHBACK implements NonLuxury {
    private String name;

    public NonLuxuryHATCHBACK(String sName) {
        name = sName;
    }

    public String getNLName() {
        return name;
    }

    public String getNLFeatures() {
        return "Non-Luxury HATCHBACK Features ";
    };
}
]]>
</file>

<file path="NonLuxurySUV.java">
<![CDATA[
public class NonLuxurySUV implements NonLuxury {
    private String name;
    public NonLuxurySUV(String sName) {
        name = sName;
    }
    public String getNLName() {

```

```

        return name;
    }
    public String getNLFeatures() {
        return "Non-Luxury SUV Features ";
    };
} // End of class
]]>
</file>

<file path="SUVVehicleFactory.java">
<![CDATA[
public class SUVVehicleFactory extends VehicleFactory {

    public LuxurySUV getLuxury() {
        return new LuxurySUV("Luxury-SUV");
    }
    public NonLuxurySUV getNonLuxury() {
        return new NonLuxurySUV("Non-Luxury-SUV");
    }
} // End of class
]]>
</file>

```

```

<file path="VehicleFactory.java">
<![CDATA[
public abstract class VehicleFactory {
    public static final String CAR = "Car";
    public static final String SUV = "Suv";
    public static final String HATCHBACK = "Hatchback";

    public abstract Luxury getLuxury();
    public abstract NonLuxury getNonLuxury();

    public static VehicleFactory getVehicleFactory(String type) {
        if (type.equals(VehicleFactory.CAR))
            return new CARVehicleFactory();
        if (type.equals(VehicleFactory.SUV))

```



```

        return new SUVVehicleFactory();
    if (type.equals(VehicleFactory.HATCHBACK)) {
        return new HATCHBACKVehicleFactory();
    }

    return new CARVehicleFactory();
}

} // End of class
]]>
</file>
</files>
</java_code>

```

The target programming language is:

```

<target_language>
Python
</target_language>

```

```

<instructions>

```

1. Carefully read and analyze the provided Java code to fully understand its logic and functionality.
2. Plan how to translate the code into Python, considering differences in syntax, paradigms, and standard libraries.
3. Preserve the original behavior and logic of the Java code while adapting it to Python conventions.
4. Write clean, concise, and idiomatic Python code (follow PEP8 guidelines).
5. Replace Java-specific constructs with appropriate Python equivalents (e.g., collections, loops, exception handling).
6. Include comments to explain non-trivial logic or important transformations.
7. Use meaningful variable and function names adapted to Python naming conventions (snake_case).
8. Simplify or improve the code where appropriate, avoiding unnecessary verbosity typical of Java.
9. Handle potential edge cases and ensure the translated code is robust.
10. If the Java code uses object-oriented design, appropriately map it to Python classes and structures.

```

</instructions>

```

Please follow these instructions when generating the translated code:

<response>

1. Begin with a brief explanation of your approach to translating the code.
2. Provide the translated Python code inside <java_code> tags.
3. After the code, include a brief explanation of how the Python version works and any important design decisions or adaptations made.

</response>

If any part of the Java code is unclear or requires assumptions, state them clearly before presenting your solution.

2.3 CÓDIGO TRANSFORMADO (PYTHON)

```
"""
```

```
Abstract Factory Pattern - Vehicle Search Application  
A Python translation of the Java AutoSearchUI application using tkinter.  
"""
```

```
import tkinter as tk  
from tkinter import ttk  
from abc import ABC, abstractmethod  
from typing import Optional
```

```
# ===== Abstract Factory =====
```

```
class VehicleFactory(ABC):  
    """Abstract factory for creating vehicle objects."""
```

```
    CAR = "Car"  
    SUV = "Suv"  
    HATCHBACK = "Hatchback"
```

```
    @abstractmethod  
    def get_luxury(self) -> 'Luxury':  
        """Create a luxury vehicle."""  
        pass
```

```
    @abstractmethod  
    def get_non_luxury(self) -> 'NonLuxury':  
        """Create a non-luxury vehicle."""
```

```

pass

@staticmethod
def get_vehicle_factory(vehicle_type: str) -> 'VehicleFactory':
    """Factory method to get the appropriate concrete factory."""
    if vehicle_type == VehicleFactory.CAR:
        return CARVehicleFactory()
    elif vehicle_type == VehicleFactory.SUV:
        return SUVVehicleFactory()
    elif vehicle_type == VehicleFactory.HATCHBACK:
        return HATCHBACKVehicleFactory()
    return CARVehicleFactory() # Default

# ===== Concrete Factories =====

class CARVehicleFactory(VehicleFactory):
    """Concrete factory for CAR vehicles."""

    def get_luxury(self) -> 'LuxuryCAR':
        return LuxuryCAR("Luxury-Car")

    def get_non_luxury(self) -> 'NonLuxuryCAR':
        return NonLuxuryCAR("NonLuxury-Car")

class SUVVehicleFactory(VehicleFactory):
    """Concrete factory for SUV vehicles."""

    def get_luxury(self) -> 'LuxurySUV':
        return LuxurySUV("Luxury-SUV")

    def get_non_luxury(self) -> 'NonLuxurySUV':
        return NonLuxurySUV("Non-Luxury-SUV")

class HATCHBACKVehicleFactory(VehicleFactory):
    """Concrete factory for HATCHBACK vehicles."""

    def get_luxury(self) -> 'LuxuryHATCHBACK':
        return LuxuryHATCHBACK("Hatchback-Luxury")

    def get_non_luxury(self) -> 'NonLuxuryHATCHBACK':
        return NonLuxuryHATCHBACK("Hatchback-NonLuxury")

# ===== Product Interfaces (Abstract Base Classes) =====

```

```

class Luxury(ABC):
    """Interface for luxury vehicles."""

    @abstractmethod
    def get_luxury_name(self) -> str:
        pass

    @abstractmethod
    def get_luxury_features(self) -> str:
        pass

class NonLuxury(ABC):
    """Interface for non-luxury vehicles."""

    @abstractmethod
    def get_nl_name(self) -> str:
        pass

    @abstractmethod
    def get_nl_features(self) -> str:
        pass

```

===== Concrete Luxury Products =====

```

class LuxuryCAR(Luxury):
    """Concrete luxury CAR product."""

    def __init__(self, name: str):
        self._name = name

    def get_luxury_name(self) -> str:
        return self._name

    def get_luxury_features(self) -> str:
        return "Luxury Car Features"

class LuxurySUV(Luxury):
    """Concrete luxury SUV product."""

    def __init__(self, name: str):
        self._name = name

    def get_luxury_name(self) -> str:

```

```

        return self._name

    def get_luxury_features(self) -> str:
        return "Luxury SUV Features"

class LuxuryHATCHBACK(Luxury):
    """Concrete luxury HATCHBACK product."""

    def __init__(self, name: str):
        self._name = name

    def get_luxury_name(self) -> str:
        return self._name

    def get_luxury_features(self) -> str:
        return "Luxury HATCHBACK Features"

# ===== Concrete Non-Luxury Products =====

class NonLuxuryCAR(NonLuxury):
    """Concrete non-luxury CAR product."""

    def __init__(self, name: str):
        self._name = name

    def get_nl_name(self) -> str:
        return self._name

    def get_nl_features(self) -> str:
        return "Non-Luxury CAR Features"

class NonLuxurySUV(NonLuxury):
    """Concrete non-luxury SUV product."""

    def __init__(self, name: str):
        self._name = name

    def get_nl_name(self) -> str:
        return self._name

    def get_nl_features(self) -> str:
        return "Non-Luxury SUV Features"

```

```

class NonLuxuryHATCHBACK(NonLuxury):
    """Concrete non-luxury HATCHBACK product."""

    def __init__(self, name: str):
        self._name = name

    def get_nl_name(self) -> str:
        return self._name

    def get_nl_features(self) -> str:
        return "Non-Luxury HATCHBACK Features"

# ===== Main GUI Application =====

class AutoSearchUI:
    """Main GUI application using tkinter."""

    LUXURY = "Luxury"
    NON_LUXURY = "Non-Luxury"
    SEARCH = "Search"
    EXIT = "Exit"

    def __init__(self):
        self._root = tk.Tk()
        self._root.title("Abstract Factory - Example")
        self._root.geometry("1050x600")

        # Handle window close event
        self._root.protocol("WM_DELETE_WINDOW", self._on_exit)

        # Create GUI components
        self._create_widgets()
        self._layout_widgets()

    def _create_widgets(self):
        """Create all GUI widgets."""
        # Category combobox
        self._cmb_vehicle_category = ttk.Combobox(
            self._root,
            values=[self.LUXURY, self.NON_LUXURY],
            state="readonly"
        )
        self._cmb_vehicle_category.set(self.LUXURY)

        # Vehicle type combobox
        self._cmb_vehicle_type = ttk.Combobox(

```

```

        self._root,
        values=[VehicleFactory.CAR, VehicleFactory.SUV, VehicleFactory.HATCHBACK],
        state="readonly"
    )
    self._cmb_vehicle_type.set(VehicleFactory.CAR)

# Labels
self._lbl_vehicle_category = ttk.Label(self._root, text="Vehicle Category:")
self._lbl_vehicle_type = ttk.Label(self._root, text="Vehicle Type:")
self._lbl_car_name = ttk.Label(self._root, text="Search Result:")
self._lbl_car_name_value = ttk.Label(
    self._root,
    text=" Please click on Search button"
)

# Buttons
self._btn_search = ttk.Button(
    self._root,
    text=self.SEARCH,
    command=self._on_search
)
self._btn_exit = ttk.Button(
    self._root,
    text=self.EXIT,
    command=self._on_exit
)

def _layout_widgets(self):
    """Layout widgets using grid geometry manager."""
    # Configure grid padding
    pad_options = {'padx': 5, 'pady': 5}

    # Row 0: Vehicle Category
    self._lbl_vehicle_category.grid(row=0, column=0, sticky='e', **pad_options)
    self._cmb_vehicle_category.grid(row=0, column=1, sticky='w', **pad_options)

    # Row 1: Vehicle Type
    self._lbl_vehicle_type.grid(row=1, column=0, sticky='e', **pad_options)
    self._cmb_vehicle_type.grid(row=1, column=1, sticky='w', **pad_options)

    # Row 2: Search Result label
    self._lbl_car_name.grid(row=2, column=0, sticky='e', **pad_options)
    self._lbl_car_name_value.grid(row=2, column=1, sticky='w', **pad_options)

    # Row 3: Buttons (with extra top padding)
    button_pad = {'padx': 2, 'pady': (40, 5)}
    self._btn_search.grid(row=3, column=0, sticky='e', **button_pad)

```

```

self._btn_exit.grid(row=3, column=1, sticky='w', **button_pad)

# Configure column weights for proper resizing
self._root.columnconfigure(0, weight=1)
self._root.columnconfigure(1, weight=1)

def get_selected_category(self) -> str:
    """Get the selected vehicle category."""
    return self._cmb_vehicle_category.get()

def get_selected_type(self) -> str:
    """Get the selected vehicle type."""
    return self._cmb_vehicle_type.get()

def set_result(self, search_result: str):
    """Update the search result label."""
    self._lbl_car_name_value.config(text=search_result)

def _on_search(self):
    """Handle search button click."""
    vh_category = self.get_selected_category()
    vh_type = self.get_selected_type()

    # Get the appropriate vehicle factory
    vf = VehicleFactory.get_vehicle_factory(vh_type)

    if vh_category == self.LUXURY:
        luxury_vehicle = vf.get_luxury()
        search_result = (f"Name: {luxury_vehicle.get_luxury_name()} "
                        f"Features: {luxury_vehicle.get_luxury_features()}")
    else: # NON_LUXURY
        non_luxury_vehicle = vf.get_non_luxury()
        search_result = (f"Name: {non_luxury_vehicle.get_nl_name()} "
                        f"Features: {non_luxury_vehicle.get_nl_features()}")

    self.set_result(search_result)

def _on_exit(self):
    """Handle exit button click or window close."""
    self._root.quit()
    self._root.destroy()

def run(self):
    """Start the GUI application."""
    self._root.mainloop()

```



```
# ===== Main Entry Point =====

def main():
    """Main entry point of the application."""
    app = AutoSearchUI()
    app.run()

if __name__ == "__main__":
    main()
```

2.4 ANÁLISIS DE RESULTADO OBTENIDO

Al poner en ejecución el código entregado por la IA obtenemos el siguiente resultado:

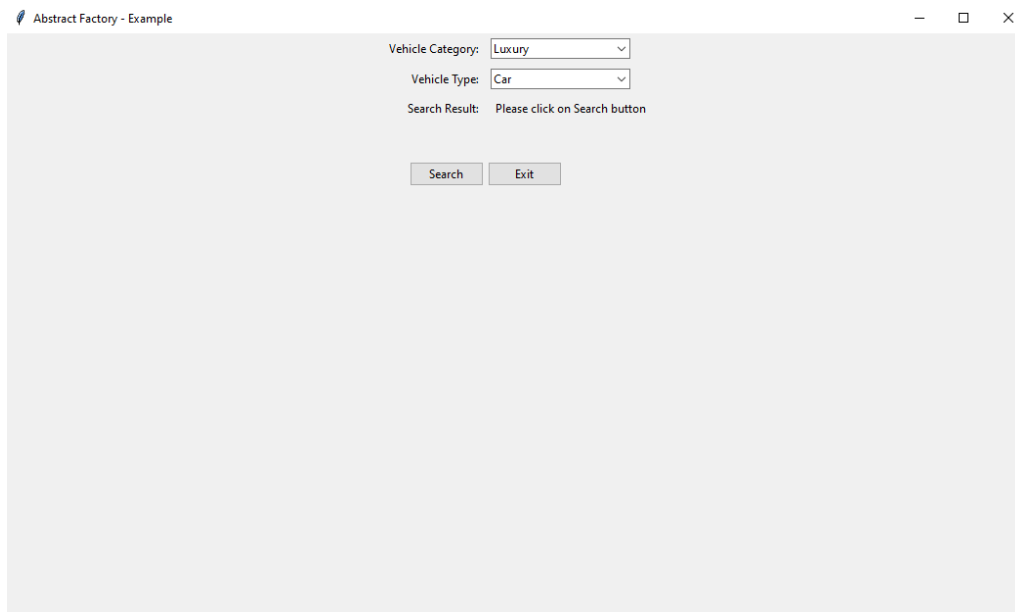


Figura 1: GUI en Python

Haciendo uso de la aplicación entregada y comparando con la versión original desarrollada en java. Se evidencia que la herramienta pudo reproducir de forma idéntica la funcionalidad y la gran mayoría de características de la interfaz gráfica de usuario en una sola iteración y en un tiempo menor a dos minutos. A nivel de construcción de código, se observa cómo la IA pudo hallar las herramientas equivalentes para la generación de elementos gráficos y para la aplicación del paradigma orientado a objetos, esto último se puede observar al momento de

definir las clases abstractas e interfaces, o ver como se emplea la herencia y el polimorfismo en python.

La herramienta mapeó con exactitud las estructuras del código original y logró adaptarla en la sintaxis de python manteniendo las buenas prácticas propias del lenguaje o como se especificó en el prompt bajo el estándar PEP.

Algo que identificó la IA y se puede evidenciar en los comentarios que realiza es el uso Patrón Fábrica Abstracta, resultó útil al momento de revisar el código transformado ya que al presentar los elementos identificados, agrupados y comentados hacía más fácil la lectura del código.

Puntos en contra identificados en el resultado obtenido:

1. No hizo separación de las estructuras en archivos lo cual hace que el programa sea un documento extenso y complicaba la lectura.
2. La interfaz gráfica si bien mantuvo todos los elementos agrupados correctamente no se dispusieron en el espacio de la misma forma como en el programa original.

Lo anterior no afecta la funcionalidad por lo cual no se realizaron más iteraciones pero de hacerlas no debería suponer un problema

2.5 REFLEXIÓN FINAL

El patrón de diseño con el que se puso a prueba el modelo de IA era uno que contenía una cantidad considerable de estructuras y de lógica de programación en comparación con otros patrones presentados a lo largo del curso, por lo que antes de realizar el ejercicio consideré que era muy grande el código dada la capacidad de contexto que tienen normalmente los modelos de IA en capa gratuita y la necesaria para tener un buen resultado, motivo clave por el que elegí DeepSeek. Pero al considerar lo eficiente que fue en este caso la transformación de código en comparación a un procedimiento manual. Podría asegurar que la herramienta puede soportar una carga más grande de información para hacer un proceso de transformación de código.

En una situación real donde se requiera este tipo de producción de código como lo puede ser una migración de tecnologías, lo veo como una alternativa productiva y eficiente pero con el nivel apropiado de escepticismo por lo que aún así debe ser

supervisada, dado que aún con versiones empresariales de herramientas de IA se puede caer en mal código causado por alucinaciones o desconocimiento de de la IA en algunas herramientas por lo que el conocimiento técnico en la creación de software sigue siendo necesaria por parte de las personas que se desempeñan en el área de TI.

3 RECUPERACIÓN DE DISEÑO

3.1 DESCRIPCIÓN DE PLATAFORMAS DE IA GENERATIVA, PROGRAMACIÓN Y EJECUCIÓN DE PROGRAMAS

Al igual que en el ejercicio anterior, se usó el modelo de IA DeepSeek V3.2

3.2 “PROMPT” UTILIZADO PARA LA TRANSFORMACIÓN DE CÓDIGO

You are an AI assistant specialized in software design recovery. Your task is to analyze the provided source code and reconstruct design artifacts strictly based on the implementation.

Your goal is to generate:

A functional model (Use Case Diagram)

A structural model (Class Diagram)

A set of Sequence Diagrams (one per use case)

Here is the source code to analyze:

```
<java_code>
  <project_structure>
    AutoSearchUI.java
    CARVehicleFactory.java
    HATCHBACKVehicleFactory.java
    Luxury.java
    LuxuryCAR.java
    LuxuryHATCHBACK.java
    LuxurySUV.java
    NonLuxury.java
    NonLuxuryCAR.java
    NonLuxuryHATCHBACK.java
    NonLuxurySUV.java
    SUVVehicleFactory.java
    VehicleFactory.java
```

```

</project_structure>
<files>
  <file path="AutoSearchUI.java">
    <![CDATA[
import java.io.*;
import java.io.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import com.sun.java.swing.plaf.windows.*;

public class AutoSearchUI extends JFrame {
    public static final String newline = "\n";
    public static final String SEARCH = "Search";
    public static final String EXIT = "Exit";
    public static final String LUXURY = "Luxury";
    public static final String NON_LUXURY = "Non-Luxury";

    private JComboBox cmbVehicleCategory, cmbVehicleType;

    private JLabel lblVehicleCategory, lblVehicleType,
        lblCarName, lblCarNameValue;

    public AutoSearchUI() {
        super("Abstract Factory - Example");

        cmbVehicleCategory = new JComboBox();
        cmbVehicleCategory.addItem(AutoSearchUI.LUXURY);
        cmbVehicleCategory.addItem(AutoSearchUI.NON_LUXURY);

        cmbVehicleType = new JComboBox();
        cmbVehicleType.addItem(VehicleFactory.CAR);
        cmbVehicleType.addItem(VehicleFactory.SUV);
        cmbVehicleType.addItem(VehicleFactory.HATCHBACK);

        lblVehicleCategory = new JLabel("Vehicle Category:");
        lblVehicleType = new JLabel("Vehicle Type:");
        lblCarName = new JLabel("Search Result:");
        lblCarNameValue = new JLabel(" Please click on Search button");
    }
}

```

```

// Create the open button
JButton openButton = new JButton(AutoSearchUI.SEARCH);
openButton.setMnemonic(KeyEvent.VK_S);
JButton exitButton = new JButton(AutoSearchUI.EXIT);
exitButton.setMnemonic(KeyEvent.VK_X);

ButtonHandler objButtonHandler = new ButtonHandler(this);

openButton.addActionListener(objButtonHandler);
exitButton.addActionListener(new ButtonHandler());

JPanel buttonPanel = new JPanel();

// *****
GridBagLayout gridbag = new GridBagLayout();
buttonPanel.setLayout(gridbag);
GridBagConstraints gbc = new GridBagConstraints();

buttonPanel.add(lblVehicleCategory);
buttonPanel.add(cmbVehicleCategory);
buttonPanel.add(lblVehicleType);
buttonPanel.add(cmbVehicleType);
buttonPanel.add(lblCarName);
buttonPanel.add(lblCarNameValue);

buttonPanel.add(openButton);
buttonPanel.add(exitButton);

gbc.insets.top = 5;
gbc.insets.bottom = 5;
gbc.insets.left = 5;
gbc.insets.right = 5;

gbc.anchor = GridBagConstraints.EAST;
gbc.gridx = 0;
gbc.gridy = 0;
gridbag.setConstraints(lblVehicleCategory, gbc);
gbc.anchor = GridBagConstraints.WEST;

```

```

gbc.gridx = 1;
gbc.gridy = 0;
gridbag.setConstraints(cmbVehicleCategory, gbc);
gbc.anchor = GridBagConstraints.EAST;
gbc.gridx = 0;
gbc.gridy = 1;
gridbag.setConstraints(lblVehicleType, gbc);
gbc.anchor = GridBagConstraints.WEST;
gbc.gridx = 1;
gbc.gridy = 1;
gridbag.setConstraints(cmbVehicleType, gbc);
gbc.anchor = GridBagConstraints.EAST;
gbc.gridx = 0;
gbc.gridy = 2;
gridbag.setConstraints(lblCarName, gbc);
gbc.anchor = GridBagConstraints.WEST;
gbc.gridx = 1;
gbc.gridy = 2;
gridbag.setConstraints(lblCarNameValue, gbc);

gbc.insets.left = 2;
gbc.insets.right = 2;
gbc.insets.top = 40;
gbc.anchor = GridBagConstraints.EAST;

gbc.gridx = 0;
gbc.gridy = 5;
gridbag.setConstraints(openButton, gbc);
gbc.anchor = GridBagConstraints.WEST;
gbc.gridx = 1;
gbc.gridy = 5;
gridbag.setConstraints(exitButton, gbc);

// *****

// Add the buttons and the log to the frame
Container contentPane = getContentPane();

```

```

        contentPane.add(buttonPanel, BorderLayout.CENTER);
    try {
        // UIManager.setLookAndFeel(new WindowsLookAndFeel());
        SwingUtilities.updateComponentTreeUI(
            AutoSearchUI.this);
    } catch (Exception ex) {
        System.out.println(ex);
    }
}

public static void main(String[] args) {
    JFrame frame = new AutoSearchUI();

    frame.addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });

    // frame.pack();
    frame.setSize(1050, 600);
    frame.setVisible(true);
}

public String getSelectedCategory() {
    return (String) cmbVehicleCategory.getSelectedItem();
}

public String getSelectedType() {
    return (String) cmbVehicleType.getSelectedItem();
}

public void setResult(String searchResult) {
    lblCarNameValue.setText(searchResult);
}
} // End of class AutoSearchUI

class ButtonHandler implements ActionListener {

```



```

AutoSearchUI objAutoSearchUI;

public ButtonHandler() {
}

public ButtonHandler(AutoSearchUI inObjAutoSearchUI) {
    objAutoSearchUI = inObjAutoSearchUI;
}

public void actionPerformed(ActionEvent e) {

    String searchResult = null;

    if (e.getActionCommand().equals(AutoSearchUI.EXIT)) {
        System.exit(1);
    }
    if (e.getActionCommand().equals(AutoSearchUI.SEARCH)) {
        // get input values
        String vhCategory = objAutoSearchUI.getSelectedCategory();
        String vhType = objAutoSearchUI.getSelectedType();

        // get one of Luxury or NonLuxury vehicle factories
        VehicleFactory vf = VehicleFactory.getVehicleFactory(vhType);

        if (vhCategory.equals(AutoSearchUI.LUXURY)) {
            Luxury c = vf.getLuxury();
            searchResult = "Name: " + c.getLuxuryName() + " Features: " +
                c.getLuxuryFeatures();
        }
        if (vhCategory.equals(AutoSearchUI.NON_LUXURY)) {
            NonLuxury s = vf.getNonLuxury();
            searchResult = "Name: " + s.getNLName() + " Features: " +
                s.getNLFeatures();
        }
        objAutoSearchUI.setResult(searchResult);
    }
}

```

```

} // End of class ButtonHandler
]]>
</file>

```

```

<file path="CARVehicleFactory.java">
<![CDATA[
public class CARVehicleFactory extends VehicleFactory {

    public LuxuryCAR getLuxury() {
        return new LuxuryCAR("Luxury-Car");
    }
    public NonLuxuryCAR getNonLuxury() {
        return new NonLuxuryCAR("NonLuxuryL-Car");
    }
} // End of class
]]>
</file>

```

```

<file path="HATCHBACKVehicleFactory.java">
<![CDATA[
public class HATCHBACKVehicleFactory extends VehicleFactory {

    public LuxuryHATCHBACK getLuxury() {
        return new LuxuryHATCHBACK("Hatchback-Luxury");
    }
    public NonLuxuryHATCHBACK getNonLuxury() {
        return new NonLuxuryHATCHBACK("Hatchback-NonLuxury");
    }
} // End of class
]]>
</file>

```

```

<file path="Luxury.java">
<![CDATA[
public interface Luxury {
    public String getLuxuryName();
    public String getLuxuryFeatures();

} // End of class

```

```
]]>
</file>
```

```
    <file path="LuxuryCAR.java">
    <![CDATA[
public class LuxuryCAR implements Luxury {
    private String name;
    public LuxuryCAR(String cName) {
        name = cName;
    }
    public String getLuxuryName() {
        return name;
    }
    public String getLuxuryFeatures() {
        return "Luxury Car Features ";
    };

} // End of class
]]>
</file>
```

```
    <file path="LuxuryHATCHBACK.java">
    <![CDATA[
public class LuxuryHATCHBACK implements Luxury {

    private String name;
    public LuxuryHATCHBACK(String cName) {
        name = cName;
    }
    public String getLuxuryName() {
        return name;
    }
    public String getLuxuryFeatures() {
        return "Luxury HATCHBACK Features ";
    };

}

]]>
</file>
```

```

        <file path="LuxurySUV.java">
        <![CDATA[
public class LuxurySUV implements Luxury {
    private String name;
    public LuxurySUV(String sName) {
        name = sName;
    }
    public String getLuxuryName() {
        return name;
    }
    public String getLuxuryFeatures() {
        return "Luxury SUV Features ";
    };
} // End of class
    ]]>
</file>

```

```

        <file path="NonLuxury.java">
        <![CDATA[
public interface NonLuxury {
    public String getNLName();
    public String getNLFeatures();
} // End of class
    ]]>
</file>

```

```

        <file path="NonLuxuryCAR.java">
        <![CDATA[
public class NonLuxuryCAR implements NonLuxury {
    private String name;
    public NonLuxuryCAR(String sName) {
        name = sName;
    }
    public String getNLName() {
        return name;
    }
}

```

```

    public String getNLFeatures() {
        return "Non-Luxury CAR Features ";
    };

} // End of class
]]>
</file>

<file path="NonLuxuryHATCHBACK.java">
<![CDATA[
public class NonLuxuryHATCHBACK implements NonLuxury {
    private String name;

    public NonLuxuryHATCHBACK(String sName) {
        name = sName;
    }

    public String getNLName() {
        return name;
    }

    public String getNLFeatures() {
        return "Non-Luxury HATCHBACK Features ";
    };
}
]]>
</file>

<file path="NonLuxurySUV.java">
<![CDATA[
public class NonLuxurySUV implements NonLuxury {
    private String name;
    public NonLuxurySUV(String sName) {
        name = sName;
    }
    public String getNLName() {
        return name;
    }
    public String getNLFeatures() {

```

```

        return "Non-Luxury SUV Features ";
    };

} // End of class
    ]]>
</file>

    <file path="SUVVehicleFactory.java">
    <![CDATA[
public class SUVVehicleFactory extends VehicleFactory {

    public LuxurySUV getLuxury() {
        return new LuxurySUV("Luxury-SUV");
    }
    public NonLuxurySUV getNonLuxury() {
        return new NonLuxurySUV("Non-Luxury-SUV");
    }
} // End of class
    ]]>
</file>

```

```

    <file path="VehicleFactory.java">
    <![CDATA[
public abstract class VehicleFactory {
    public static final String CAR = "Car";
    public static final String SUV = "Suv";
    public static final String HATCHBACK = "Hatchback";

    public abstract Luxury getLuxury();
    public abstract NonLuxury getNonLuxury();

    public static VehicleFactory getVehicleFactory(String type) {
        if (type.equals(VehicleFactory.CAR))
            return new CARVehicleFactory();
        if (type.equals(VehicleFactory.SUV))
            return new SUVVehicleFactory();
        if (type.equals(VehicleFactory.HATCHBACK)) {
            return new HATCHBACKVehicleFactory();
        }
    }
}
    ]]>
</file>

```

```

    }

    return new CARVehicleFactory();
}

} // End of class
]]>
</file>
</files>
</java_code>

```

Please follow these strict instructions:

<instructions>

1. Source of Truth:

- The source code is the ONLY valid source.
 - Do NOT infer, assume, or invent functionality, actors, relationships, or behaviors.

2. Functional Model (Use Case Diagram):

- Extract use cases ONLY from real execution points
- Actors must be strictly derived from observable interactions in the code.
- Do NOT invent actors or unnecessary use cases.
- Output in PlantUML.

3. Structural Model (Class Diagram):

- Extract all classes, interfaces, attributes, and methods exactly as defined in the code.

- Relationships MUST be identified and represented with HIGH PRECISION:

a. Inheritance / Implementation:

- Use ONLY when "extends" or "implements" is explicitly present.

b. Association:

- Only if a class holds a reference to another class (attribute, parameter, or persistent usage).
- MUST include direction if identifiable.

- c. Dependency:
 - When a class uses another class temporarily (e.g., local variable, method call, parameter usage).
- d. Aggregation / Composition:
 - Only if clearly deducible from code (e.g., strong ownership via attributes and lifecycle hints).
 - If not स्पष्ट, DO NOT assume composition.
- Cardinality (MANDATORY):
 - Infer multiplicity ONLY when it is explicitly or strongly implied:
 - Collections (e.g., List, Array, JComboBox) → 0..*, 1..*
 - Single object references → 1 or 0..1 depending on initialization
 - If multiplicity is unclear → DO NOT invent, omit it instead.
- Do NOT add any relationship that is not directly supported by the code.
- Output in PlantUML.
- 4. Sequence Diagrams:
 - For EACH use case, generate EXACTLY ONE sequence diagram.
 - STRICT TRACEABILITY:
 - Every message MUST correspond to a real method call in the code.
 - Every participant MUST map to an actual class/object.
 - Flow:
 - Reflect the real execution order as implemented in the code.
 - Respect control structures (if, method calls, object creation).
 - Object Creation:
 - Include object instantiation ONLY if explicitly present (e.g., "new ClassName()").
 - DO NOT:
 - Invent methods
 - Invent flows
 - Simplify logic beyond what exists

6. Naming:

- Use EXACT names from the code.
- No renaming, no abstraction.

7. Output Format:

- ALL diagrams MUST be valid PlantUML.
- Keep diagrams separated and clearly labeled.

</instructions>

Provide your response in the following format:

<response>

<use_case_diagram>

@startuml

' PlantUML code here

@enduml

</use_case_diagram>

<class_diagram>

@startuml

' Must include:

' - Correct relationships

' - Correct types (inheritance, association, dependency)

' - Cardinalities ONLY when valid

' - No invented links

@enduml

</class_diagram>

<sequence_diagrams>

<sequence_diagram name="UseCase_1_Name">

@startuml

' Strictly traceable interactions

@enduml

</sequence_diagram>

</sequence_diagrams>

</response>

3.4 ANÁLISIS COMPARATIVO DE LOS MODELOS ENTREGADOS DE LA IA

3.4.1 DIAGRAMA FUNCIONAL

El diagrama entregado por la IA es el siguiente:

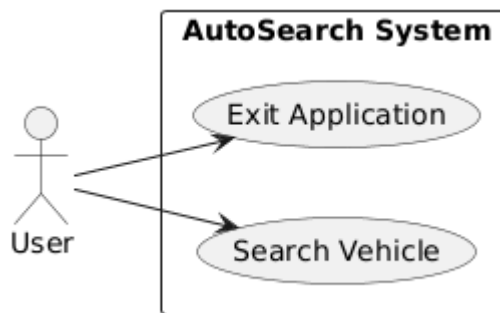


Figura 2: Diagramas de Casos de Uso Propuesto por la IA

Que en contraste con el realizado para la asignatura

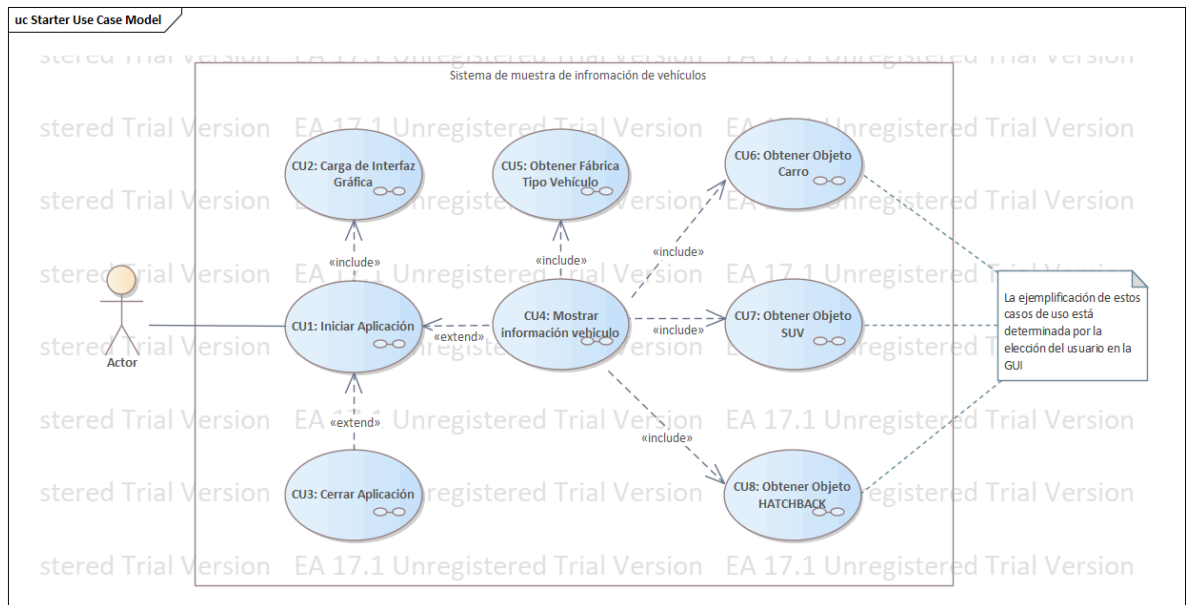


Figura 3: Diagramas de Casos de Uso Creado en la Asignatura

Se observa que la IA propone unos casos de uso con granularidad muy alta y se está basando en aspectos muy observables a nivel de interacción con la aplicación. No ahonda en casos de uso que pueden ser de ingeniería y otras funcionalidades que van de acuerdo al patrón Fábrica Abstracta. El resultado es técnicamente válido desde la perspectiva de lo que un usuario externo puede desencadenar directamente, pero pierde la riqueza del diseño arquitectónico subyacente que el ejercicio buscaba recuperar.

3.4.2 DIAGRAMA ESTRUCTURAL

El diagrama entregado por la IA es el siguiente

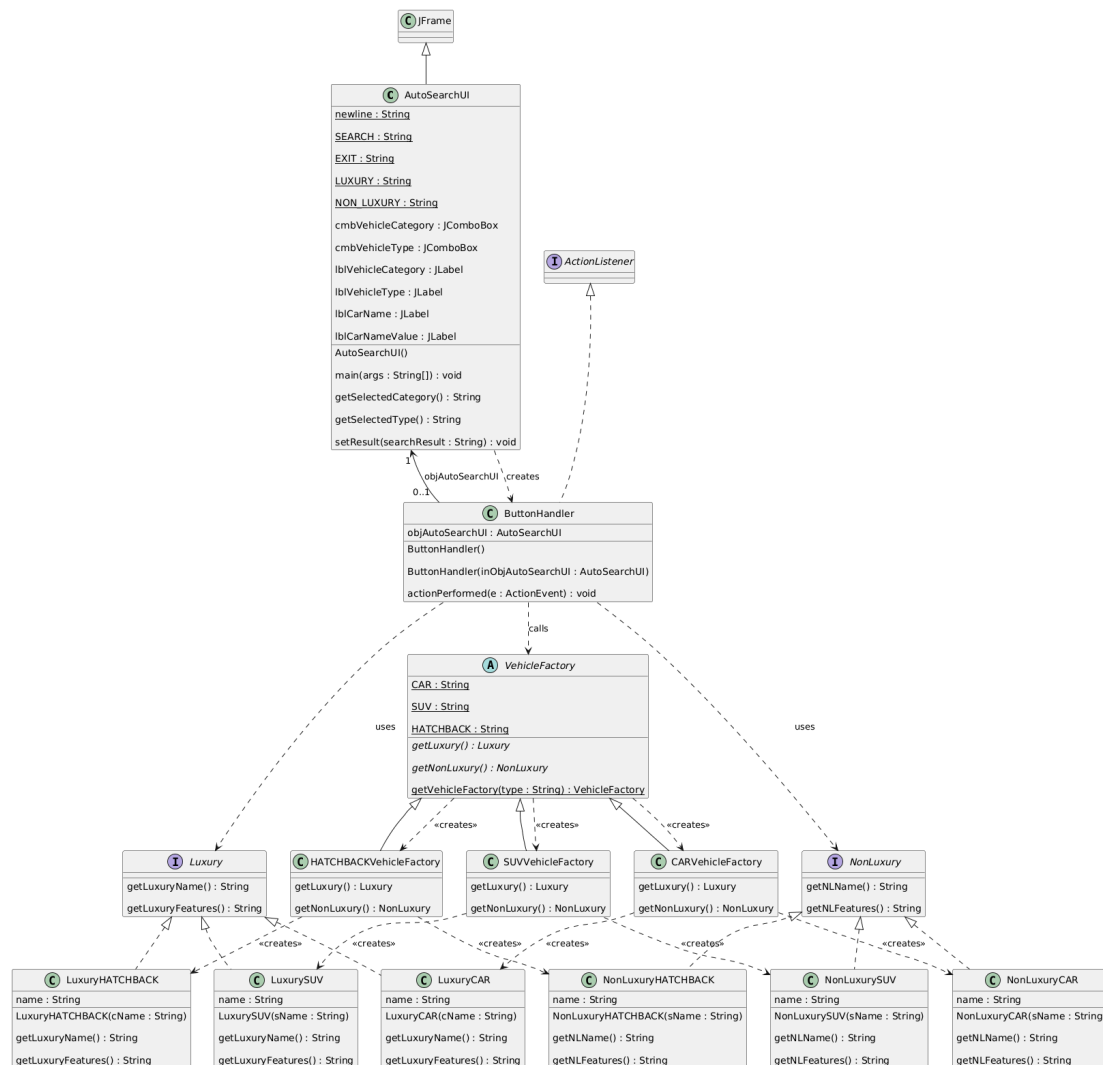


Figura 4: Diagramas de Clases Generado por la IA

En contraste con lo realizado en la asignatura

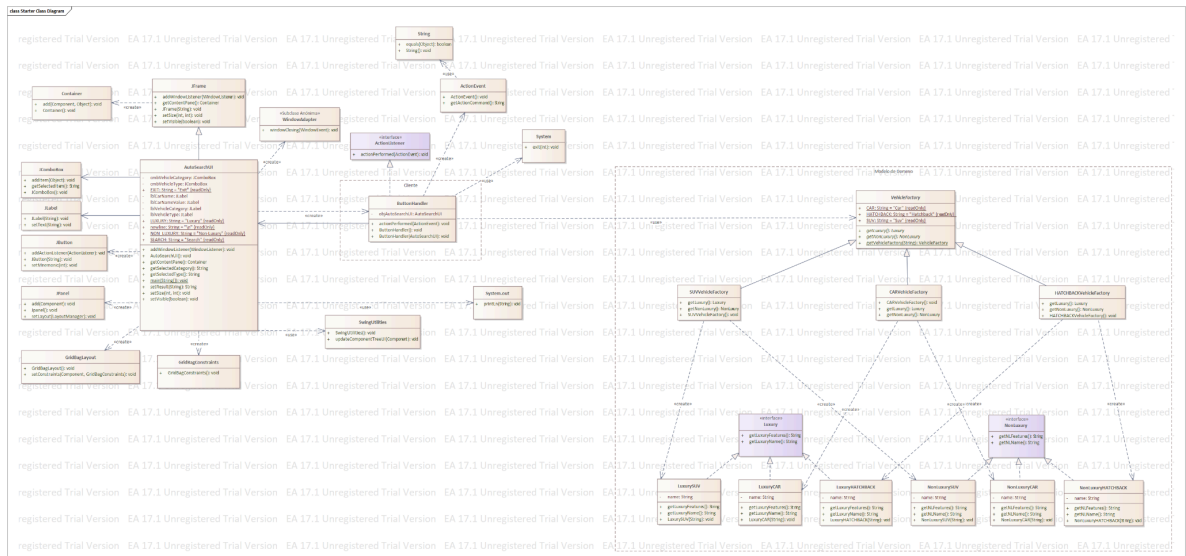


Figura 5: Diagramas de Clases Creado en la Asignatura

La IA en este caso realizó una recuperación de diseño que si bien no es estrictamente detallada sigue siendo válida, esto puede deberse a que construir el diagrama de clases no requiere de un ejercicio de inferencia ni de análisis complejo ya que solo requiere de un proceso de interpretación a nivel de código de las estructuras que compone el programa y sus relaciones. Ambos diagramas coinciden en la estructura jerárquica del patrón la fábrica abstracta en el centro, las fábricas concretas especializándose por tipo de vehículo, y los productos concretos agrupados bajo las interfaces, para este diagrama se tuvo que hacer una iteración adicional dado que en el primer intento no compilaba el código entregado en plantUML

3.4.3 DIAGRAMA DE SECUENCIA

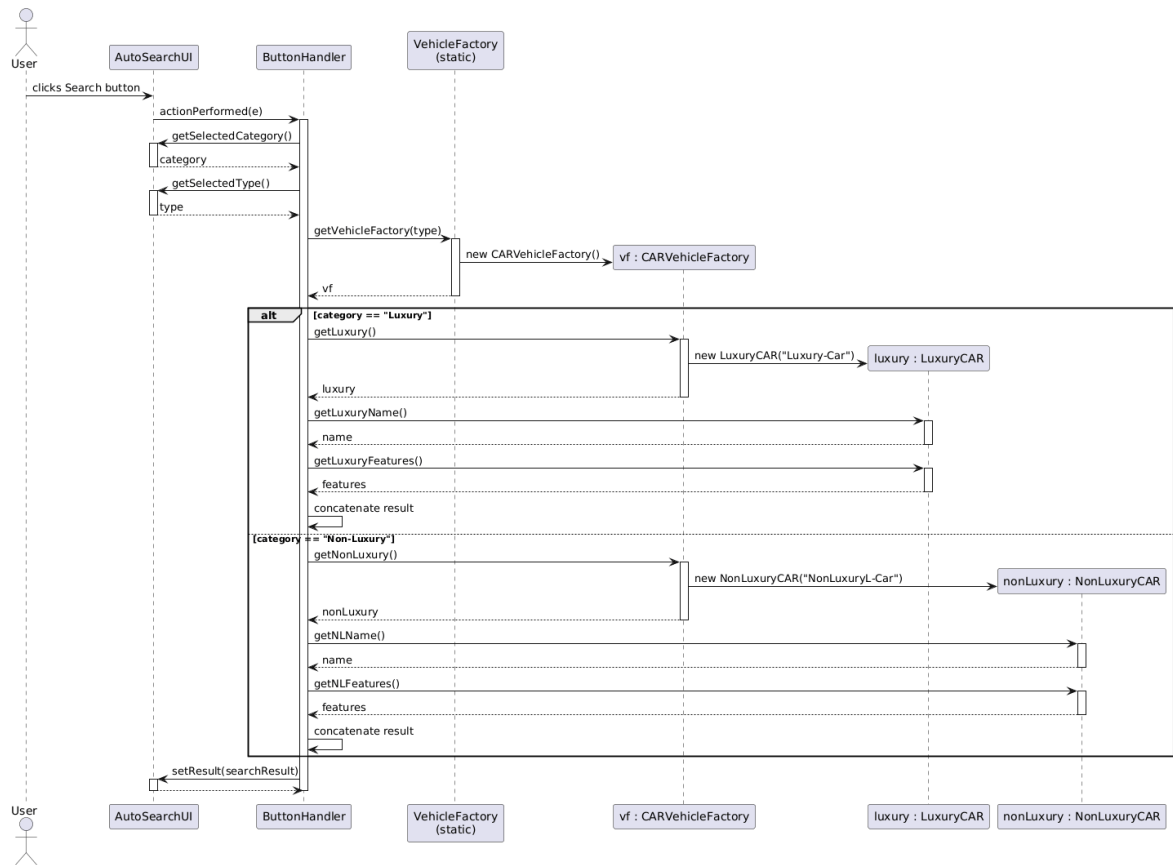


Figura 6: Diagrama de Secuencia generado por la IA/ CU “search vehicle”

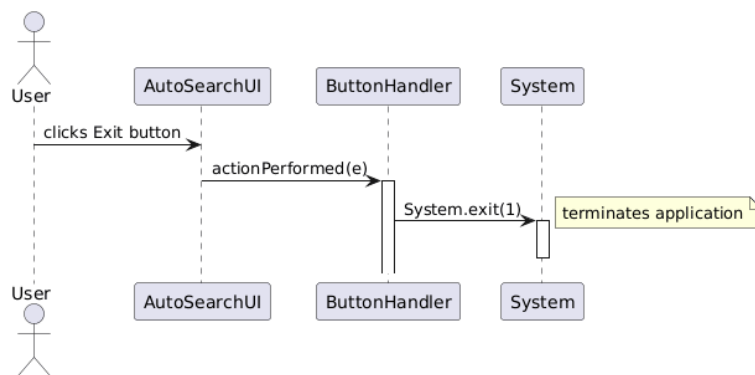


Figura 7: Diagrama de Secuencia generado por la IA/ CU “Exit Application”

Dado que la IA identificó únicamente dos casos de uso en el diagrama funcional, omitiendo aspectos como la inicialización de la interfaz gráfica y la creación de manejadores de eventos, los diagramas de secuencia resultantes carecen de

consistencia temporal: no hay claridad sobre en qué momento del ciclo de vida de la aplicación ocurre cada interacción, ni se usan referencias entre diagramas para establecer esa continuidad. Adicionalmente, en el caso de uso de búsqueda se evidencia un mal manejo del polimorfismo, ya que el diagrama fija un único tipo concreto de fábrica y producto en lugar de representar la variabilidad que el patrón garantiza en tiempo de ejecución, lo cual se refleja directamente en el mal uso del fragmento combinado “alt”, que debería capturar no sólo la distinción entre categorías sino también la variación por tipo de vehículo, pero en cambio simplifica el flujo a un único escenario presentándose como si fuera el comportamiento general del sistema.

3.5 REFLEXIÓN FINAL

Con lo evidenciado en la recuperación de diseño realizada por el modelo de IA, se puede concluir que su capacidad para recuperar el diseño a partir de código fuente es considerablemente limitado aun cuando el prompt está bien estructurado y las instrucciones son específicas. La IA demostró que puede con tareas de generación de diagramas de clase con algo de dificultad y falta de detalle. Aún así, identificó correctamente todas las clases, sus relaciones, los tipos de vínculo entre ellas y el flujo de ejecución, produciendo diagramas en PlantUML válidos y coherentes con la implementación.

Sin embargo, el ejercicio también expone una limitación importante: la IA tiende a operar en el nivel de lo explícitamente visible en el código, sin proyectar decisiones de diseño de nivel superior que un arquitecto humano sí reconocería. Esto se evidenció con mayor claridad en el diagrama funcional, donde la IA entregó un resultado pobre frente al elaborado manualmente, en los diagramas de secuencia deja mucho que desear por el nivel de detalle en situaciones como la creación de objetos o plantear o cómo llevar a cabo el caso de uso propuesto por la IA misma.

En un contexto empresarial, no recomendaría el uso de IA como propuesta de recuperación de diseño ni ingeniería directa dado que es pobre aún en tareas donde se requiere análisis fuerte y una alta comprensión del contexto en el que se está elaborando modelos que representan software construido o por construir